So far we have studied FS and FT. Summarizing the properties —

Time domain		Frequency domain
Continuous + Periodic $x(t) = \sum_{k=-\infty}^{\infty} X[k\omega_0]\, e^{jk\omega_0 t}$	CT FS ⟷	Discrete + Aperiodic $X[k\omega_0] = \frac{1}{T}\int_0^T x(t)\cdot e^{-jk\omega_0 t}\, dt$
Discrete + Periodic $x[n] = \sum_{k=0}^{N-1} X[k\Omega_0]\, e^{jk\Omega_0 n}$	DT FS ⟷	Discrete + Periodic $X[k\Omega_0] = \frac{1}{N}\sum_{n=0}^{N-1} x[n]\cdot e^{-jk\Omega_0 n}$
Continuous + Aperiodic $x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(\omega)\, e^{j\omega t}\, d\omega$	CT FT ⟷	Continuous + Aperiodic $X(\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t}\, dt$
Discrete + Aperiodic $x[n] = \frac{1}{2\pi}\int_{-\pi}^{\pi} X(\omega)\, e^{j\omega n}\, d\omega$	DT FT ⟷	Continuous + Periodic $X(\omega) = \sum_{n=-\infty}^{\infty} x[n] e^{j\omega n}$

Since we will be working with discrete signals in real life, DTFT is of our main concern. But the continuous Fourier domain is definitely a problem. We cannot store continuous signals and so we cannot store the frequency response. So we need to sample the frequency domain. But once again, the process should be reversible, and so we cannot sample the frequency domain freely. So what can we do?

Is there a transformation that does land to a discrete frequency response among all the transformations we have learnt? Yes there is. Its the DTFS. But FS only works for periodic signals and we are trying to apply Fourier on aperiodic signals.

So what can we do? Aperiodic signals generally have limited span. In fact, all real world signals span for a limited time. So, in theory we can assume all real world signals as periodic signals by repeating the signal from $-\infty$ to $+\infty$. Then we can apply DTFS on all DT signals. This formulation is commonly referred to as DFT (Discrete Fourier Transformation). So DFT and DTFS are the same (almost same). Just our motivation is different.

For any periodic discrete signal, DTFS is given by —

$$X[k\Omega_0] = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \cdot e^{-jk\Omega_0 n}$$

$$x[n] = \sum_{k=0}^{N-1} X[k\Omega_0]\, e^{jk\Omega_0 n}$$

And for any aperiodic discrete signal, DFT is given by —

$$X[k\Omega_0] = \sum_{n=0}^{N-1} x[n] \cdot e^{-jk\Omega_0 n}$$

Just the scaling factor is removed just like we did for all FTs

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k\Omega_0]\, e^{jk\Omega_0 n}$$

here, $\Omega_0 = \frac{2\pi}{N}$ ← length of the original signal

Note that the frequency resolution exactly the same as the time resolution for DFT.

Computational complexity of DFT —

$$X[k\Omega_0] = \sum_{n=0}^{N-1} x[n] \cdot e^{-jk\Omega_0 n}$$

$Len(X) = N$

for each k value there is a multiplication between a real and a complex number. This requires 2 operations $(a \cdot (a+jb) = aa + jab)$. We are multiplying N times. So, number of computations so far $= 2N$.

After the multiplication we have summation between N complex summations. It takes 2 (N-1) computations.

So, n(operations) $= 2N + 2N - 2 = 4N - 2$

We are ignoring the evaluation overhead of $e^{-jk\Omega_0 n}$ because of all the symmetries. It is generally precalculated and if not, the complexity is O(N). These factors are called twiddle factors and they depend only on N. Exploiting the symmetries can significantly reduce the number of computations of twiddle factors.

Extending over N k values we get n(operations) $= 4N_2 - 2N + O(N)$ (→ twiddle overhead)

So complexity of DFT is $O(n^2)$.

Can we do better? Assume N is 2^c and $N > 1$

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-jk\Omega_0 n}$$

$$= \underbrace{\sum_{n=0}^{N/2-1} x[2n] e^{-jk\Omega_0 \cdot 2n}}_{\text{Even indices}} + \underbrace{\sum_{n=0}^{N/2-1} x[2n+1] e^{-jk\Omega_0 (2n+1)}}_{\text{Odd indices}}$$

$$= \sum_{n=0}^{N/2-1} x[2n] e^{-jk\Omega_0 \cdot 2n} + e^{-jk\Omega_0} \sum_{n=0}^{N/2-1} x[2n+1] e^{-jk\Omega_0 \cdot 2n}$$

What is Ω_0? $\Omega_0 = \frac{2\pi}{N}$. Then $2\Omega_0 = \frac{2\pi}{N/2} = \Omega_0'$

also lets define $x[2n]$ as $x_e[n]$, a new signal with length $= N/2$

in the same way, $x[2n+1] = x_0[n]$, once again with length $= N/2$

This is possible because $N = 2^e$

So essentially $X[k] = \sum_{n=0}^{N/2-1} x_e[n] e^{-jk\Omega_0' n} + e^{-jk\Omega_0} \sum_{n=0}^{N/2-1} x_0[n] e^{-jk\Omega_0' n}$

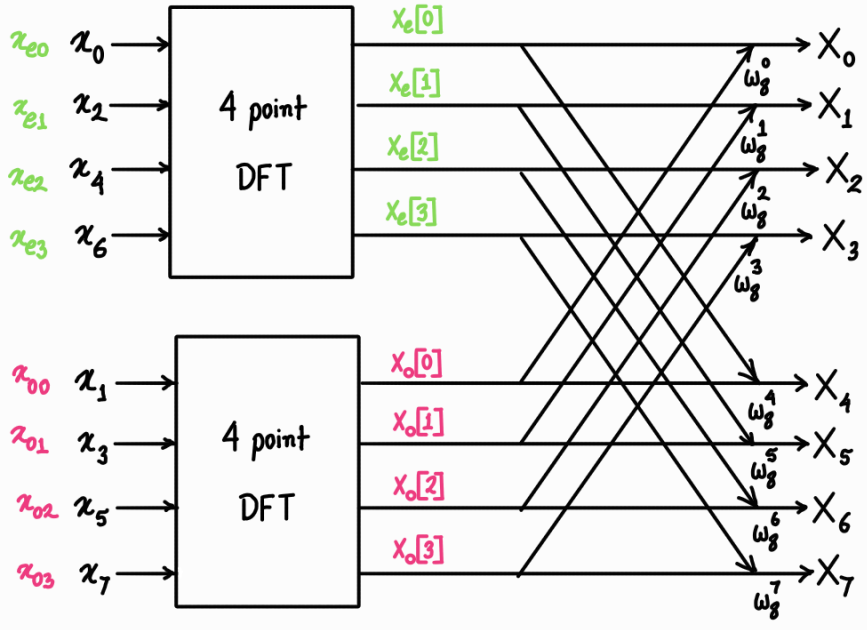
This is interesting because $\sum_{n=0}^{N/2-1} x_e[n] e^{-jk\Omega_0' n} = DFT_{N/2}(x_e)$

and $e^{-jk\Omega_0} \sum_{n=0}^{N/2-1} x_0[n] e^{-jk\Omega_0' n} = e^{-jk\Omega_0} DFT_{N/2}(x_0)$

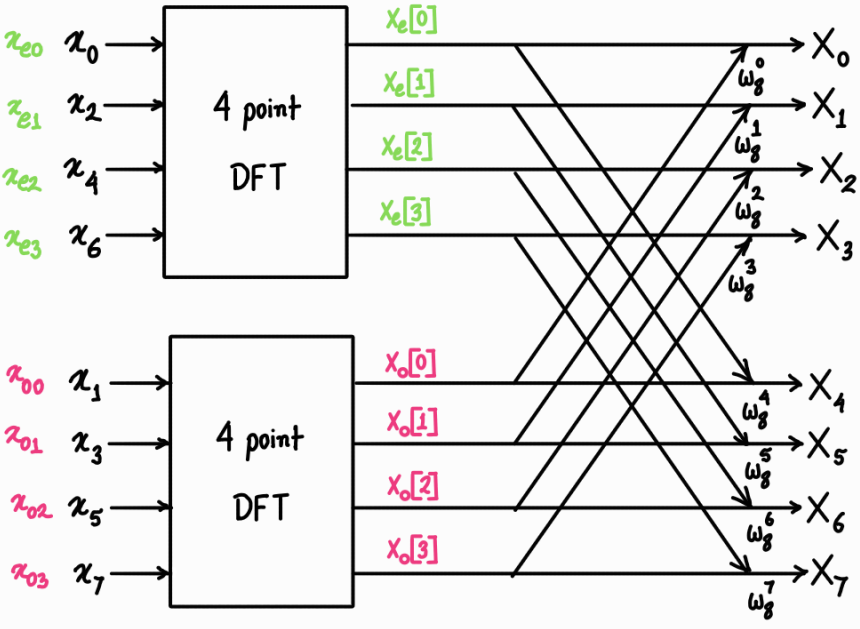
Do you see the possibility of recursion? Lets prepare a data flow graph and consider $e^{-jk\Omega_0} = W_N^k$ for easier representation. We will draw the graph for $N = 8$.

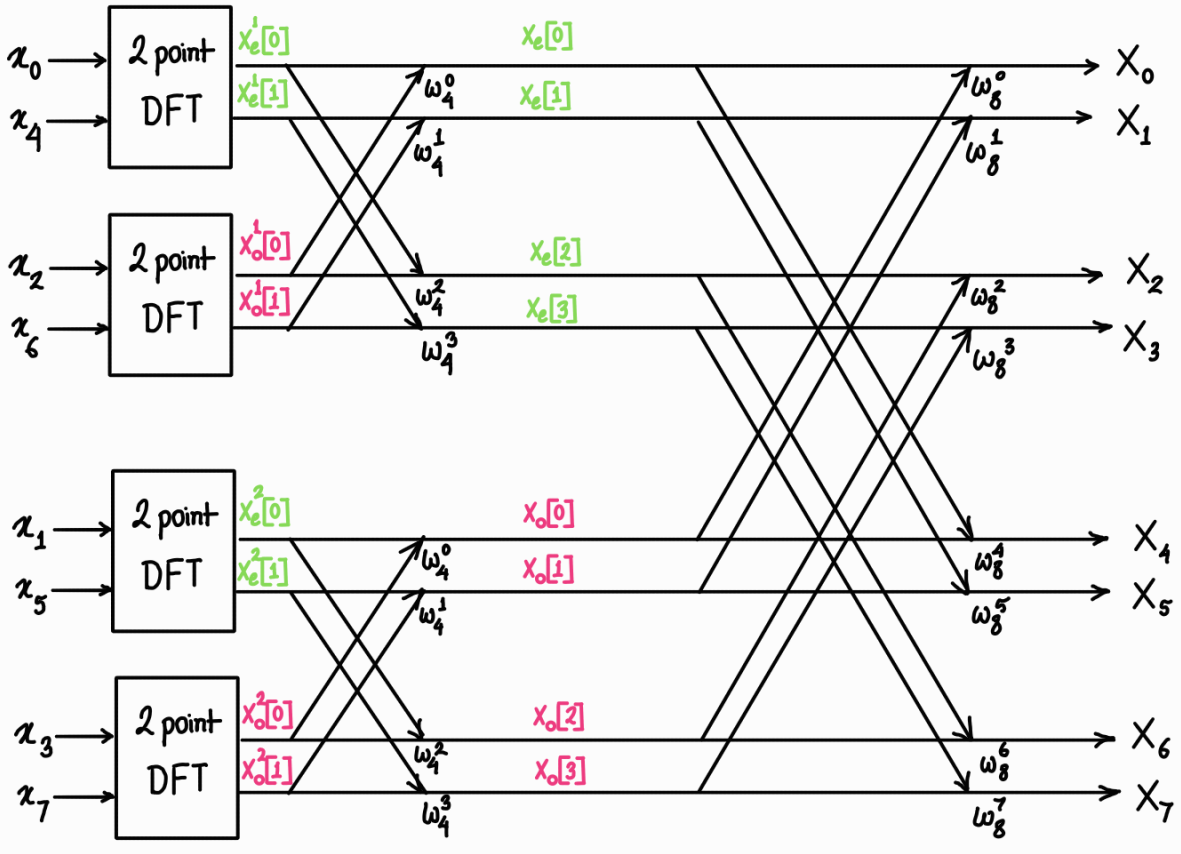
$$X[k] = DFT_{N/2}(x_e) + W_N^k DFT_{N/2}(x_0) = X_e[k] + W_N^k X_0[k]$$

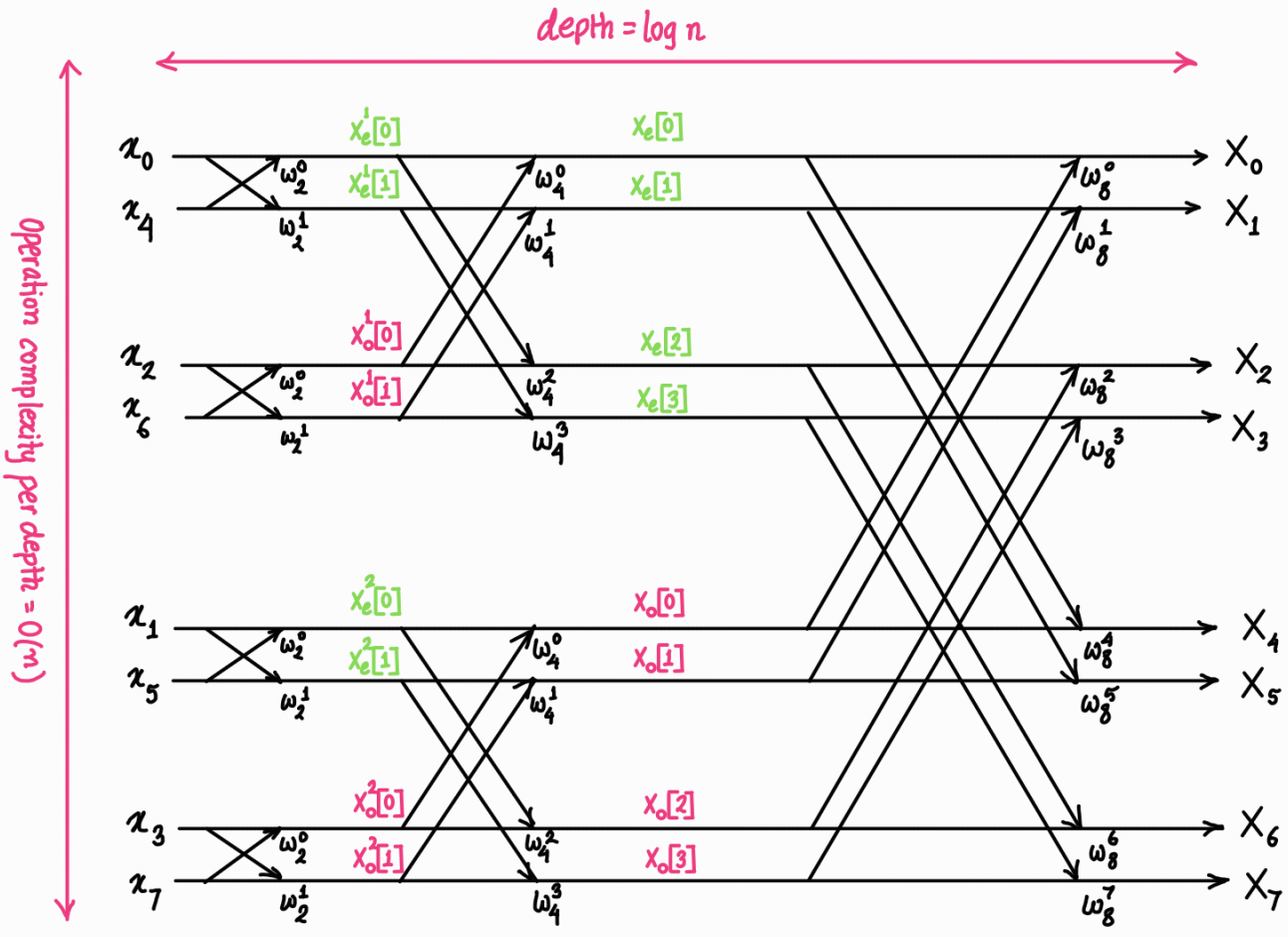
Please note that the formula might feel broken because X_e and X_0 should only have 4 points indexed from 0 to 3 but k is indexed from 0 to 7. The formula works because according to derivation X_e and X_0 are periodic and $X_e[4] = X_e[0]$



Now this can be extended further —







1 point DFT $X[0] = x[0] \cdot e^0 = x[0]$

So, $X[0] = x[0] \cdot 1$

Pseudocode of FFT —

```
def fft(x):
    n = len(x)
    x = zero_pad(x, mode = 'right', n = pow(2, ceil(log2(n))) - n )
    n = len(x)
    if n == 1:
        return 1
    W = exp(-2πj/n)
    X_even, X_odd = fft(x[::2]), fft(x[1::2])
    X = [0] * n
    for k in range(n/2):
        X[k] = X_even[k] + W^k · X_odd[k]
        X[k+n/2] = X_even[k] - W^k · X_odd[k]
                          └→ a bit different
    return X
```

Prove $W^{K+N/2} = -W^K$ —

$$W_N^K = e^{-j\frac{2\pi}{N}K}$$

then $$W_N^{K+N/2} = e^{-j\frac{2\pi}{N}(K+N/2)}$$

$$= e^{-j\frac{2\pi}{N}K} \cdot e^{-j\pi}$$

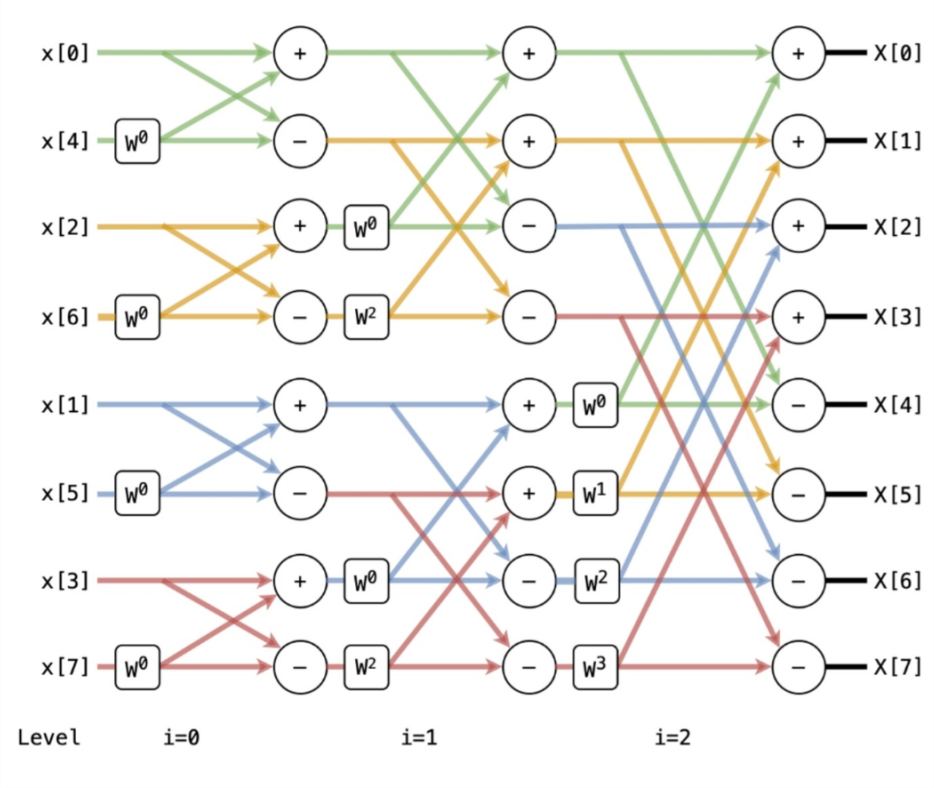
$$= (\cos\pi - j\sin\pi) W^K = -W^K$$

The FFT data flow diagram is also called a butterfly diagram.

Pseudocode of IFFT —

```
def ifft(X):
    n = len(X)
    X = zero_pad(X, mode = 'right', n = pow(2, ceil(log2(n))) - n )
    n = len(X)
    if n == 1:
        return 1
    W = exp(2πj/n)
    x_even, x_odd = ifft(X[::2]), ifft(X[1::2])
    x = [0] * n
    for k in range (n/2):
        x[k] = x_even[k] + W^k · x_odd[k]
        x[k+n/2] = x_even[k] - W^k · x_odd[k]
    return x
def scale_ifft(x):
    n = len(x)
    return iff(x)/n
```

Find the changes.



Fully optimized 8 input FFT

(Collected from HMC)